COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	09/07/2026	**DATE SUBMITTED:**	12/07/2026
NAME:	Ignacio Guillian Carlo	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands on ACTIVITY NO. 1.1 [Hands-on Activity 1.1 Basic C++ Programming]

1. PROCEDURE OUTPUT

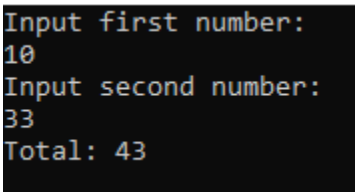
PART A - 1

Sections Answer	**ANSWER**
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	`void sotn(int Fnum, int Lnum);`
Class Declaration and Method Definition Section	NONE

Main Function	`int main(){`
	`  int Fnum;`
	`  int Lnum;`
	`  std::cout << "Input first number: " << std::endl;`
	`  std::cin >> Fnum;`
	`  std::cout << "Input second number: " << std::endl;`
	`  std::cin >> Lnum;`
	`  sotn(Fnum, Lnum);`
	`  return 0;`
	`}`

Method Definition	`void sotn(int Fnum, int Lnum) {`
	`  int Total = Fnum + Lnum;`
	`  std::cout << "Total: "<< Total << std::endl;`
	`}`

Table 1.1. C++ Structure Code for Answer



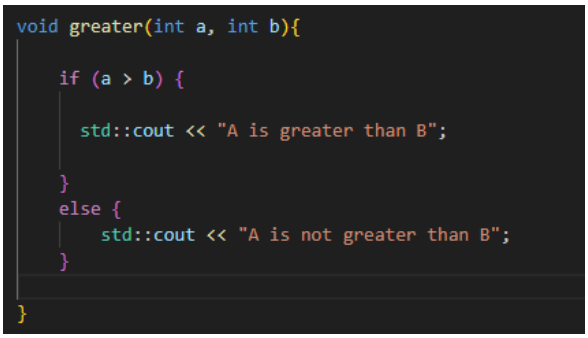
The output for this one is very simple. I used a user input, I had problems earlier since I was putting the user input inside the void function when it should be put inside the main function. I have 2 integer variables, Fnum and Lnum. I first declared the function at the top of the int main() function and at the bottom I declared there what the function is supposed to do. Which is to add Fnum and Lnum after the user inputs the values for it. Once the user is done inputting the values it will output the total as shown in the picture where I put 10 as the first number and 33 as the second number which adds to 43.

Table 1.2. **ILO A output observations and comments.**

PART A - 2

SECTION	**ANSWER**
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	`void greater(int a, int b);`
Class Declaration and Method Definition Section	NONE

Main Function	`int main(){`
	`  int a;`
	`  int b;`
	
	`  std::cout << "Input First Number: " << std::endl;`
	`  std::cin >> a;`
	`  std::cout << "Input second Number: "<< std::endl;`
	`  std::cin >> b;`
	
	`  greater(a,b);`
	
	`  return 0;`
	`}`

Method Definition	

```
void greater(int a, int b){

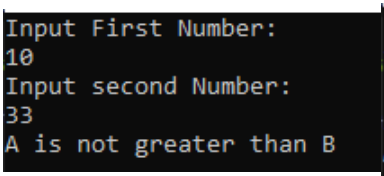
    if (a > b) {

      std::cout << "A is greater than B";

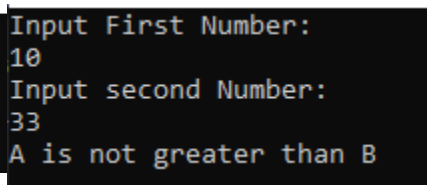
    }
    else {
        std::cout << "A is not greater than B";
    }

}
```

Table 2.1. C++ Structure Code for Answer



```
Input First Number:
10
Input second Number:
33
A is not greater than B
```

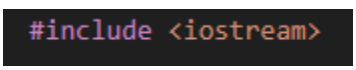
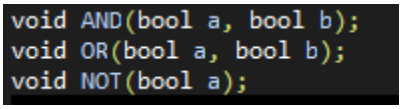


```
Input First Number:
10
Input second Number:
33
A is not greater than B
```

I made a void function named greater, and its definition which is below the int main() or driver program. What it does is determine if the value of A is greater or lesser than B, if its greater it will output “A is greater than B” if its lesser it will output A is lesser than B. I integrated user input in this one. So as we can see in the pictures above I asked the user to input the first number and second number and it will output the right decision. I made this work by using an if else statement where (a > b) If a is greater it will output that a is greater vice versa.

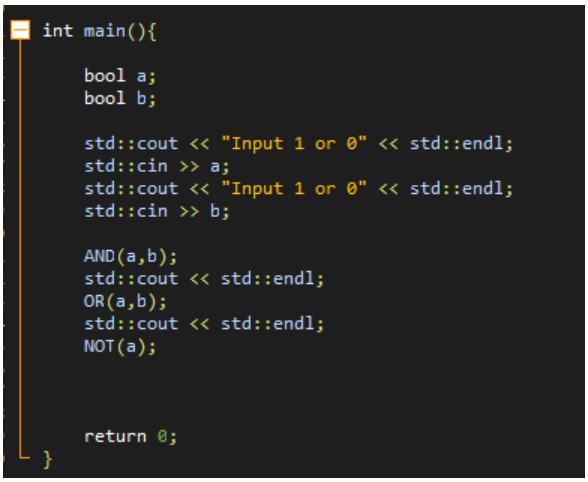
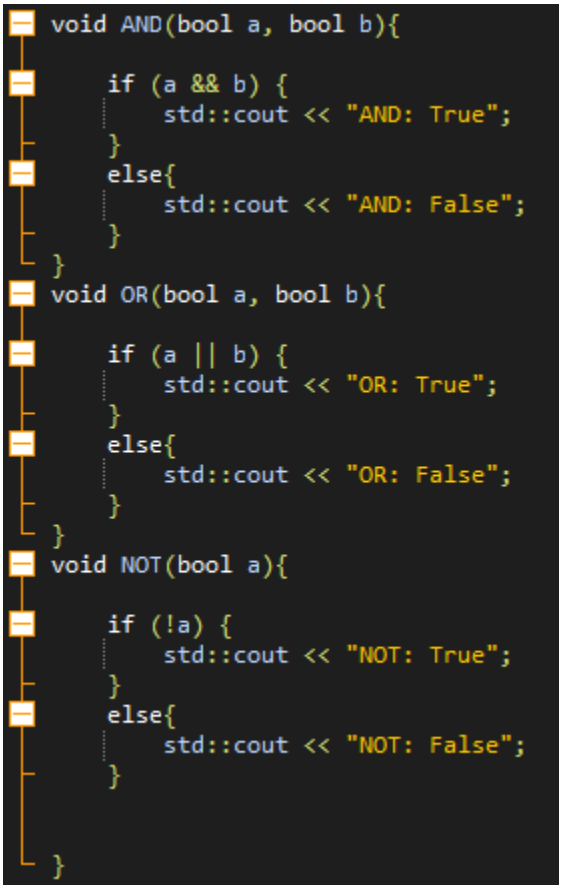
Table 2.2. **ILO A output observations and comments.**

PART A-3

SECTION	**ANSWER**
Header File Declaration Section	
Global Declaration Section	

```
#include <iostream>
```

```
void AND(bool a, bool b);
void OR(bool a, bool b);
void NOT(bool a);
```

Class Declaration and Method Definition Section	NONE
Main Function	
Method Definition	

```
int main(){

    bool a;
    bool b;

    std::cout << "Input 1 or 0" << std::endl;
    std::cin >> a;
    std::cout << "Input 1 or 0" << std::endl;
    std::cin >> b;

    AND(a,b);
    std::cout << std::endl;
    OR(a,b);
    std::cout << std::endl;
    NOT(a);



    return 0;
}
```

```
void AND(bool a, bool b){

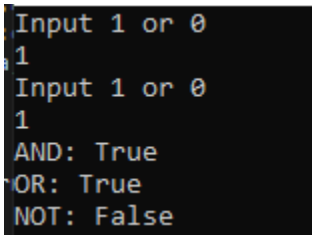
    if (a && b) {
        std::cout << "AND: True";
    }
    else{
        std::cout << "AND: False";
    }
}
void OR(bool a, bool b){

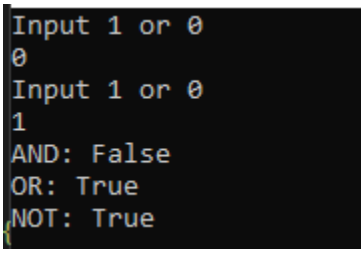
    if (a || b) {
        std::cout << "OR: True";
    }
    else{
        std::cout << "OR: False";
    }
}
void NOT(bool a){

    if (!a) {
        std::cout << "NOT: True";
    }
    else{
        std::cout << "NOT: False";
    }


}
```

Table 3.1. C++ Structure Code for Answer

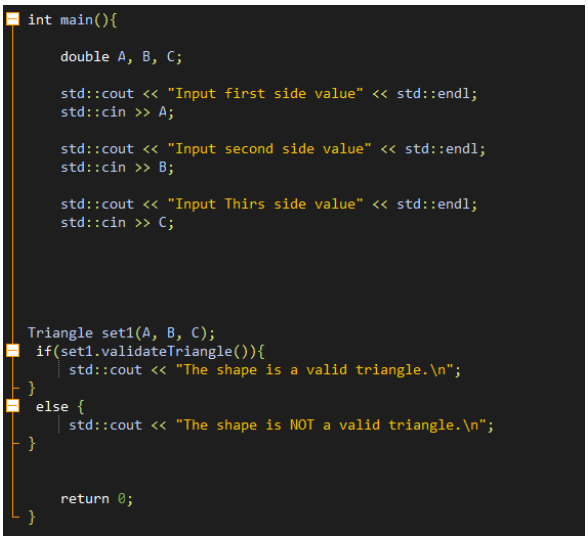
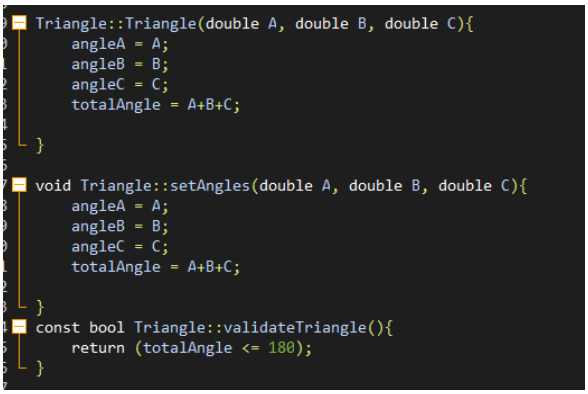




As we can see in these outputs the console will only take a boolean value if it does not the program will end immediately. To explain this code: I made 3 functions AND, OR, and NOT, I used these operators to make the code work and their symbols are &&, II, and !=. I used these in the definitions inside the functions on what they'll do so for AND basically if both the inputs are the same it will return False but if the inputs are different it will output False, for OR the output will only show false if you input 0 in both of the inputs. And for NOT, it only bases on the first number since it reverses the result so if we input 0 it will output True but in reality 0 is = false but if we input 1 it will output False since 1 is = True.

Table 3.2. **ILO A output observations and comments.**

SECTION	**ANSWER**
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	None
Class Declaration and Method Definition Section	`class Triangle{ private: double totalAngle, angleA, angleB, angleC; public: //constructor Triangle(double A, double B, double C); //Methods void setAngles(double A, double B, double C); //Validation const bool validateTriangle(); };`

Main Function	
Method Definition	

```
int main(){

    double A, B, C;

    std::cout << "Input first side value" << std::endl;
    std::cin >> A;

    std::cout << "Input second side value" << std::endl;
    std::cin >> B;

    std::cout << "Input Thirs side value" << std::endl;
    std::cin >> C;




Triangle set1(A, B, C);
 if(set1.validateTriangle()){
     std::cout << "The shape is a valid triangle.\n";
}
 else {
     std::cout << "The shape is NOT a valid triangle.\n";
}


    return 0;
}
```

```
Triangle::Triangle(double A, double B, double C){
      angleA = A;
      angleB = B;
      angleC = C;
      totalAngle = A+B+C;

}

void Triangle::setAngles(double A, double B, double C){
      angleA = A;
      angleB = B;
      angleC = C;
      totalAngle = A+B+C;

}
const bool Triangle::validateTriangle(){
      return (totalAngle <= 180);
}
```

Table 4.1. C++ Structure Code for Answer

```
Input first side value
21
Input second side value
100
Input Thirs side value
231
The shape is NOT a valid triangle.
```

I used a user input for this ILO too, basically if the values add up to more than 180 it will say that it is not a valid triangle since a triangles angle is 180 degrees. So firstly I declared the private and public classes. What private does is it is hidden to everyone inside the code only Triangle can access it while public is accessible to everyone here so the purpose of private is to avoid unnecessary or accidental changes to it. I didn't use the setAngle function that much but after trying different things with it, I believe it is a setter. So lets return to the main program, basically I just ask the user 3 values if it is less than or equal

to 180 it is a triangle but if it is more than 180 it is not a triangle.

Table 4.2. **ILO B output observations and comments.**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Create a C++ program to swap the two numbers in different variables.

```cpp
#include <iostream>
#include <utility>


int main(){

    int a;
    int b;

    std::cout << "Input first number" << std::endl;
    std::cin >> a;

    std::cout << "Input second number" << std::endl;
    std::cin >> b;

    std::swap(a, b);
    std::cout << std::endl;

    std::cout << "First number: " << a <<std::endl << "Second number: " << b << std::endl;


    return 0;
}
```

Figure 1 <Code snippet of Supplementary no.1>

For the first supplementary task I used the **Utility** header in order to use its built-in **Swap** function since the task is to just create a program that swaps two numbers of different variables. So the algorithm of this program is first to just input the first number and second number then the swap function will immediately do its job and it will output the swapped numbers.

```
Input first number
10
Input second number
21

First number: 21
Second number: 10
```

Figure 1.1 <Output snippet of supplementary task no.1>

As we can see here, the values have swapped because we used the built-in swap function of Utility header.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

```
#include <iostream>

void KtF (int K);


int main(){

    int K;

    std::cout << "Kelvin value: ";
    std::cin >> K;

    KtF(K);


    return 0;
}

void KtF (int K){

    double Fahrenheit;

    Fahrenheit = 1.8 * (K - 273.15) + 32;

    std::cout << "Fahrenheit: " << Fahrenheit;

}
```

Figure 2 <Code snippet for Supplementary no.2>

I didn't have any difficulties in making the program since it was just like the first task in ILO A, but what I did here is declare a function at the top of the int main() then named is KtF since this is a Kelvin to Fahrenheit calculator. Then after that I coded its definition below where I declared Fahrenheit first then I inputted the Kelvin to Fahrenheit formula which is 1.8 x (Kelvin - 273.15) + 32 then it will output the value. For this one I also used a user input for the value of Kelvin.

```
Kelvin value: 231.21
Fahrenheit: -43.87
```

Figure 2.1<Output snippet of Supplementary no.3>

As we can see here, if we input the value of Kelvin as 231 or what I did here 231.21 the Fahrenheit value will be -43.87

3. Create a C++ program that has a function that will calculate the distance between two points.

```
#include <iostream>
#include <cmath>

void Distance(double a, double b, double c, double d);

int main(){
    int a;
    int b;
    int c;
    int d;


    std::cout << "Input value of x1: " << std::endl;
    std::cin >> a;

    std::cout << "Input value of x2: " << std::endl;
    std::cin >> b;

    std::cout << "Input value of y1: " << std::endl;
    std::cin >> c;

    std::cout << "Input value of y2: " << std::endl;
    std::cin >> d;

    Distance(a, b, c, d);


    return 0;
}

void Distance(double a, double b, double c, double d){



    double x = b - a;
    double y = d - c;

    double distance = std::sqrt((x * x) + (y * y));

    std::cout << "The Distance is: " << distance;

```

Figure 3.1 <Code snippet for Supplementary no.3>

I didn't exactly had a hard time doing this, but I had a hard time conceptualizing how I'll perform the distance formula itself inside the code. I could use both sqrt() and pow() but I only used sqrt() for this one. It is much easier to separate the x and the y's in terms of like when we need to subtract them instead of putting them both inside the formula, so that's what I did here, I subtracted them first and declared them onto another variable now inside the distance formula I multiplied them to themselves to get the x squared and y squared then I added them up and used the std::sqrt() to finalize the answer.

```
Input value of x1:
4
Input value of x2:
6
Input value of y1:
8
Input value of y2:
10
The Distance is: 2.82843
```

Figure 3.2 <Output snippet of Supplementary no.3>

As we can see here the terminal asks the user for a value for x1, x2, y1, y2 and after that the distance formula will be calculated precisely.

4. Modify the code given in ILO B and add the following functions:

a. A function to compute for the area of a triangle

```
double Area (double base, double height);
```

```
double Triangle::Area(double base, double height){

    return 0.5 * base * height;
}
```

b. A function to compute for the perimeter of a triangle

```
double Perimeter(double A, double B, double C);
```

```
double Triangle::Perimeter(double A, double B, double C){

    return A + B + C;
}
```

c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

```
void Angle();
```

```cpp
void Triangle::Angle(){
    if(angleA < 90 && angleB < 90 && angleC < 90){
        std::cout << "Acute Triangle";
    }
    else if(angleA > 90 || angleB > 90 || angleC > 90){
        std::cout << "Obtuse Triangle";
    }
    else {
        std::cout << "Others";
    }

}
```

```
Input first side value
30
Input second side value
40
Input Thirs side value
50
Enter base: 21
Enter height: 32
The shape is a valid triangle.

Area = 336

Perimeter = 120

Acute Triangle
```

I had trouble integrating the new functions but I figured that it would make sense if I inserted them in the public class, this was by far harder than the tasks before. I had to use AI to ask how the public and private classes work and if I can reuse them instead of making a new function outside. Let's move on to the explanation of the program, So firstly I declared the function inside the public class, then defined it at the bottom, lets discuss the Area function where it returns the value after it has been calculated, I used 0.5 * base * height to calculate the area of the triangle, next for the perimeter I did the same thing in declaring and defining it at the bottom I used A + B + C to get the perimeter. As for the Angles I used logical operators to make sure they all will be in the same terms. The algorithm of this program is very simple, it will ask the user for the side values of the triangle then also its base and height, then it will calculate everything and show you the output.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CHATGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **EXPLANATION OF CONCEPTS**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **HAD TO ASK HOW PUBLIC AND PRIVATE CLASSES WORK**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.